Object Oriented Programming

Date : 01/06/2026
Page No. : 1

Introduction to OOP in Python

1. What is OOP ?

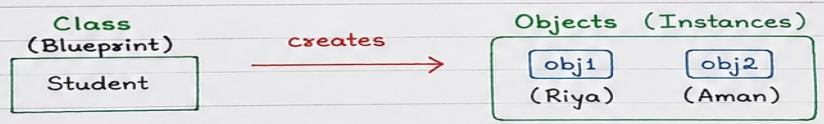
OOP (Object-Oriented Programming) is a programming paradigm based on the concept of "objects", which can contain data and code.

➢ It helps in writing code that is :

- Modular
- Reusable
- Easy to maintain

2. Class and Object

- **Class** : A blueprint or template for creating objects.
  (e.g., Student class)
- Object : An instance of a class.
  (e.g., a specific student like "Riya")



3. Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Riya", 20)
print(s1.name)    # Output: Riya
print(s1.age)     # Output: 20
```

Explanation :

- Student is a class.
- s1 is an object of Student class.
- __init__() is a constructor used to initialize object attributes.

Note : Everything in Python is an object.
e.g., int, str, list, dict, etc. are also objects.

The "self" in OOP (Part 1)

Date : 01/06/2026
Page No. : 1

1. What is "self" ?

- In Python, "self" is a reference to the current object of the class.
- It is used to access the attributes and methods of that object inside a class.
- "self" is always the first parameter in any instance method.

> Note : "self" is not a keyword.
> You can name it something else, but "self" is the convention followed.

2. Why do we need "self" ?

- When we create an object, Python needs a way to distinguish between different objects.
- "self" helps Python know which object's data to use.

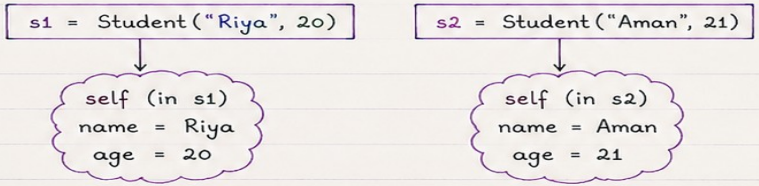
3. Example

```
class Student:                # constructor
    def __init__(self, name, age):
        self.name = name      # instance variable
        self.age  = age

    def show(self):
        print(f"Name: {self.name}, Age: {self.age}")
```

Explanation :

- self.name and self.age belong to the current object.
- self is used to access those variables.



The "self" in OOP (Part 2)

Date : 01/06/2026
Page No. : 2

4. How "self" works ?

- When you call a method using an object, Python automatically passes that object as the first argument (self).
- Inside the method, "self" refers to that object.

Example

```
class Student:                    # class
    def show(self):               # self is the current object
        print(self.name)          # access instance variable
```

```
s1 = Student("Riya", 20)
s1.show()  ⟶  Here, s1 is passed automatically as self
```

same as `s1.show(s1)`

5. Key Points

- "self" must be the first parameter in instance methods.
- You can name it anything, but "self" is the standard.
- "self" is not needed in class methods (@classmethod) and static methods (@staticmethod).
- It allows methods to work with the data of the specific object that called them.

6. Summary

- "self" = reference to the current object.
- Helps access variables (attributes) and methods of that object.
- Automatically passed when a method is called using an object.
- Essential for instance methods.

Note : Remember ⟶ self connects the method with the object that is using it.

The "__init__" in OOP (Part 1)

Date : 01/06/2026
Page No. : 1

1. What is __init__ ?

- __init__ is a special method in Python classes.
- It is called automatically when an object is created from a class.
- It is used to initialize (set) the attributes of the object.
- Also called constructor.

2. Syntax

```
def __init__(self, parameter1, parameter2, ...):
    # initialization code
```

Note : self is always the first parameter.
It refers to the current object (instance) of the class.

3. Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

This method will be called automatically when an object is created.

Here,
name and age are attributes of the object.
__init__ is used to give initial values to these.

4. Creating Object

```
s1 = Student("Riya", 20)
s2 = Student("Aman", 21)
```

→

```
s1.name → "Riya"
s1.age  → 20
s2.name → "Aman"
s2.age  → 21
```

Note : As soon as the object is created, __init__() runs automatically and sets the values.

The __init__ in OOP (Part 2)

Date : 01/06/2026
Page No. : 2

5. How __init__ works ?

- When an object is created, Python allocates memory for it.
- Then, __init__() is called automatically.
- It receives the new object (self) and any arguments you pass.
- Inside __init__(), we set the initial state (attributes) of that object.

6. Example with Explanation

```
class Student:
    def __init__(self, name, age):
        self.name = name    # name is an attribute
        self.age  = age     # age is an attribute

s1 = Student("Riya", 20)   # __init__() is called automatically
print(s1.name)    # Output: Riya
print(s1.age)     # Output: 20
```

When s1 is created, __init__() runs and sets name = "Riya" and age = 20 for that object.

7. Key Points

- __init__() is called only once, at the time of object creation.
- It can take zero or more parameters.
- If we don't define __init__(), Python provides a default constructor.
- __init__() helps in initializing the object with meaningful data.

Summary

- __init__() is a constructor in Python.
- It is called automatically when an object is created.
- The first parameter is always self.
- It is used to initialize the attributes of the object.

Note : __init__() makes our objects ready to use with proper initial values.

The __init__ in OOP (Extra)

Date : 01/06/2026
Page No. : 3

★1. Default Constructor

- A constructor that does not take any parameters (except self) is called a default constructor.
- It is used to initialize an object with default values.

```
class Student:
    def __init__(self):        # default constructor
        self.name = "Unknown"
        self.age  = 0

# Creating object
s1 = Student()    # __init__() called automatically
print(s1.name)    # Output: Unknown
print(s1.age)     # Output: 0
```

Key Point :
If we do not pass any arguments while creating an object, Python calls the default constructor.

★2. Return of the Constructor

- A constructor (__init__) never returns any value.
- Not even "None" is returned.
- It always returns None implicitly.
- We cannot use 'return' with a value in __init__().

```
class Test:
    def __init__(self):
        self.x = 10
        # return 100  ✗ (Not allowed)

t = Test()
print(t)              # Output: <__main__.Test object at 0x....>
print(t.__init__())   # Output: None
```

Why None ?
Because Python constructors are meant for initialization, not for returning values.

Summary
- Default constructor → no parameters (except self).
- Constructor (__init__) does not return any value.
- It always returns None implicitly.

CONCLUSION

1. What is __init__?

- `__init__` is a special method (constructor) in Python classes.
- It is called automatically when an object is created from a class.
- It is used to initialize (set) the attributes of the object.

2. Syntax

```
def __init__(self, param1, param2, ...):
    # initialization code
```

Note : self is always the first parameter. It refers to the current object (instance) of the class.

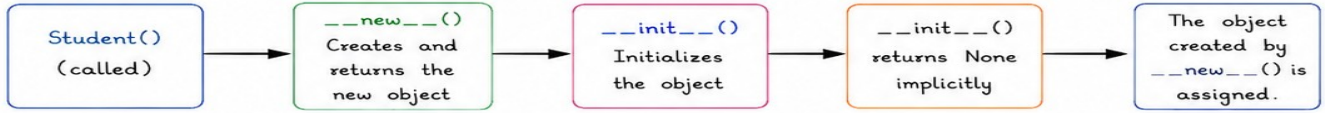
3. Default Constructor

- If a class does not define `__init__()`, Python uses the default `__init__()` inherited from the object class.
- This default constructor takes no parameters (except self) and does nothing.

4. Return of the Constructor

- A constructor (`__init__()`) never returns any value.
- Not even None is returned explicitly.
- It always returns None implicitly.
- We cannot use 'return' with a value in `__init__()`.
- We can write 'return' without a value (or nothing). It means return None.

5. How Object is Created Internally



Note : The object is returned by `__new__()`, not by `__init__()`.

6. Key Points Summary

- `__init__()` is a constructor in Python.
- It is called automatically when an object is created.
- self is the first parameter (refers to current object).
- If `__init__()` is not defined, Python uses the default `__init__()` from object class.
- `__init__()` is used to initialize attributes.

- A constructor never returns any value.
- It always returns None implicitly.
- We cannot return any value from `__init__()`.
- We can write 'return' without value (or nothing) which means return None.
- Don't confuse: object is returned by `__new__()`, not by `__init__()`.

Remember: `__init__()` → Initialize | `__new__()` → Create

Data Hiding in Python (OOP)

Date : 01/06/2026
Page No. : 4

1. What is Data Hiding ?

- Data hiding means restricting direct access to the data (attributes) of a class from outside the class.
- It is one of the basic principles of OOP which helps in security and control over data.
- In Python, data hiding is achieved by using private variables.

Why Data Hiding ?

- Protects data from accidental modification.
- Allows controlled access through methods.
- Improves code security and reliability.
- Helps in maintaining data integrity.

2. How to Achieve Data Hiding in Python ?

- In Python, we use double underscore (__) before a variable name to make it private.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance     # private variable
    def get_balance(self):
        return self.__balance        # getter method
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount    # controlled access
```

Note :

- __balance is a private variable.
- It cannot be accessed directly from outside the class.
- Access is provided only through methods (get_balance, deposit).

3. Example

```
acc = BankAccount(1000)
print(acc.get_balance())      # Output: 1000
acc.deposit(500)
print(acc.get_balance())      # Output: 1500
# Trying to access private variable directly
print(acc.__balance)
# Error: AttributeError
# 'BankAccount' object has no attribute "__balance"
```

4. Important Points

- Python does not have strict private access like other languages (e.g., Java, C++).
- It follows name mangling for private variables.
- It is a convention to treat __variable as private and access it only through methods.

Summary : Data hiding in Python is implemented using double underscores (__) before variable names and accessing them through methods.

Types of Access in Python (OOP) & Name Mangling

Date : 01/06/2026
Page No. : 5

Python does not have strict access modifiers like other languages (Java, C++).
It follows a convention based approach.

Type of Access	How to Declare	Meaning & Purpose	Access From	Example

1. Public	No underscore
	Example:
	self.name
		• Most open access.
		• Can be accessed from anywhere (inside class, outside class, subclass).
		• Used for attributes and methods that are meant to be public.
			Everywhere
			• Inside class
			• Outside class
			• Subclass
				class Person:
				def __init__(self):
				self.name = "Riya" # public
				p = Person()
				print(p.name) # Allowed
				p.name = "Seema" # Allowed

2. Protected	Single underscore (_)
	Example:
	self._age
		• Indicates "protected" members".
		• Should be accessed only inside class and subclasses.
		• It is a weak convention.
			Inside class and Subclass
			• Inside class
			• Subclass
			(Not recommended outside class)
				class Person:
				def __init__(self):
				self._age = 20 # protected
				class Student(Person):
				def show(self):
				print(self._age) # Allowed
				s = Student()
				s.show() # Allowed

3. Private	Double underscore (__)
	Example:
	self.__salary
		• Indicates "private" members".
		• Should be accessed only inside class.
		• Not accessible from outside class or subclass directly.
		• Implemented using name mangling.
			Inside class only
			• Inside class
			(Not accessible outside class or subclass directly)
				class Person:
				def __init__(self):
				self.__salary = 50000 # private
				def show(self):
				print(self.__salary) # Allowed
				p = Person()
				# print(p.__salary) # Error!

4. Name Mangling	Automatic renaming by Python
	__var → _ClassName__var
		• Python internally changes private variable names.
		• Helps to avoid accidental access and name conflicts in subclasses.
		• It is NOT encryption.
			Accessible using mangled name (Not recommended)
			_ClassName__var
				class Person:
				def __init__(self):
				self.__salary = 50000
				p = Person()
				print(p._Person__salary) # Works
				# But not recommended!

Summary Table

Type	Syntax	Access	Best Use
Public	self.name	Everywhere	General use
Protected	self._name	Inside class & subclass	For internal use (by convention)
Private	self.__name	Inside class only	To prevent accidental access
Name Mangling	__name → _Class__name	Accessible via mangled name only	Avoid conflicts in subclasses

Key Points

- Python does not have strict access modifiers.
- It follows convention based access.
- Use _ (single underscore) for protected members.
- Use __ (double underscore) for private members.
- Private members are name mangled to _ClassName__name.
- Access private members using mangled name is possible but not recommended.

Remember : Public → No underscore | Protected → _var | Private → __var → _ClassName__var (mangled)

Data Abstraction in Python (OOP)

Date : 01/06/2026
Page No. : 6

1. What is Data Abstraction ?

- Data Abstraction means showing only the essential features of an object and hiding the internal implementation.
- It helps in reducing complexity and increasing security.
- We interact with an object through well-defined methods (interface) and do not worry about how it works inside.

Analogy

- Driving a car: we use steering, accelerator, brake (interface).
- But we don't need to know how the engine, gear box or braking system works internally.
- Similarly, in abstraction we use methods and hide the internal details.

2. How is Data Abstraction Achieved in Python ?

- Python does not have keywords like abstract.
- It is achieved using classes, methods and properties.
- We expose only necessary functionalities and hide the rest (using access modifiers – _, __).

Key Point

Abstraction is about what an object does, not how it does.
Focus on interface, not implementation.

3. Example

```
class BankAccount:
    def __init__(self, owner, balance):
        self.__balance = balance    # internal detail (hidden)
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount    # access through method
    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient balance")
    def get_balance(self):
        return self.__balance    # controlled access
```

Here, the user interacts only with methods:
- ✅ deposit()
- ✅ withdraw()
- ✅ get_balance()

The internal data (__balance) is hidden and cannot be accessed directly.

Usage

```
acc = BankAccount("Riya", 1000)
acc.deposit(500)
acc.withdraw(200)
print(acc.get_balance())    # Output: 1300
# acc.__balance    --> AttributeError
# Cannot access hidden data directly
```

4. Benefits

- ★ Hides unnecessary details.
- ★ Reduces complexity.
- ★ Improves security by preventing direct access.
- ★ Makes code easier to maintain and modify.
- ★ Promotes reusability.

5. Summary

- Data Abstraction = show essential features, hide details.
- In Python, achieved using classes, methods, and access modifiers.
- We interact with objects through methods (interface).
- Focus on what the object does, not how it does.

Remember : Abstraction is the foundation of clean, secure and maintainable OOP code.

Encapsulation in Python (OOP)

Date : 01/06/2026
Page No. : 7

1. What is Encapsulation ?

- Encapsulation is the process of wrapping (binding) data and methods together in a single unit (class) and restricting direct access to some of the object's components.
- It is one of the core principles of OOP.
- It helps in data hiding and protects the internal state of an object from accidental modification.

Why Encapsulation ?

- Protects data from unintended changes. ✓
- Improves security and reliability. ✓
- Hides internal implementation details. ✓
- Provides controlled access through methods (getters/setters). ✓
- Promotes code reusability and maintainability. ✓

2. How Encapsulation is Achieved in Python ?

- Python does not have strict access modifiers like private, protected (as in Java/C++).
- It uses naming conventions and access modifiers:

Modifier	How to Write	Meaning	Access
Public	variable	No underscore	Accessible everywhere
Protected	_variable	Single underscore	Should be accessed inside class and subclasses
Private	__variable	Double underscore	Not accessible outside class (Name Mangling)

Access Levels

- Public : Can be accessed from anywhere.
- Protected : Intended for internal use and subclasses.
- Private : Intended for internal use only. Accessed indirectly through methods.

Note :

- Protected and Private are by convention, not strictly enforced.
- Still, they discourage direct external access.

3. Example

```
class Employee:
    def __init__(self, name, salary):
        self.name = name              # public
        self._department = "IT"       # protected
        self.__salary = salary        # private
    def get_salary(self):             # getter for private data
        return self.__salary
    def set_salary(self, salary):     # setter with validation
        if salary > 0:
            self.__salary = salary
        else:
            print("Invalid salary!")
```

Usage :

```
emp = Employee("Riya", 50000)
print(emp.name)          # ✓ Allowed (public)
print(emp._department)   # ⚠ Allowed (protected, by convention)
print(emp.__salary)      # ✗ Error (private)

# Accessing private data through methods
print(emp.get_salary())  # ✓ Allowed
emp.set_salary(60000)
print(emp.get_salary())  # Output: 60000
# emp.__salary = 70000   # ✗ Cannot access directly
```

Encapsulation
Internal data (__salary) is hidden and accessed only through methods.

4. Key Points

- ★ Encapsulation = binding data + methods + restricting direct access.
- ★ Achieved using underscores: _, __ (by convention).
- ★ Private members are name mangled in Python.
- ★ Use getters/setters for controlled access.
- ★ Enhances security, integrity and maintainability of code.

5. Name Mangling (for Private Members)

- When we use double underscore (__) before a variable, Python internally changes the name to:

  `_ClassName__variableName`

- Example: __salary in class Employee becomes _Employee__salary
- It prevents accidental access, not hacking.

Summary : Encapsulation in Python helps in hiding internal data and showing only what is necessary through methods. It leads to secure, clean and maintainable OOP code.

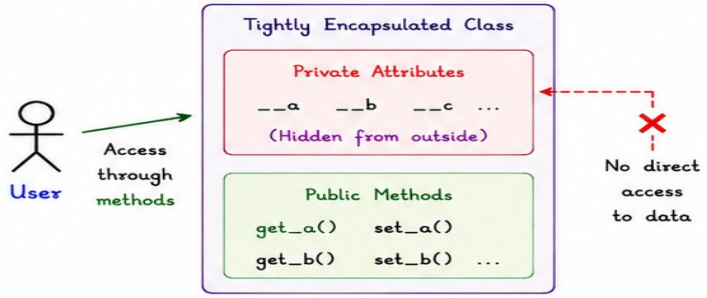
Tightly Encapsulated Class in Python

Date : 01/06/2026
Page No. : 8

1. What is Tightly Encapsulated Class ?

- A tightly encapsulated class hides all its data (attributes) and allows access only through public methods (getters/setters).
- All attributes are private (__) and cannot be accessed directly from outside the class.
- This provides maximum security and control over the data.

2. Concept Diagram



3. Example

```
class BankAccount:
    def __init__(self, owner, balance):
        self.__owner = owner          # private attribute
        self.__balance = balance      # private attribute
    def get_balance(self):
        return self.__balance         # getter method
    def set_balance(self, amount):
        if amount >= 0:
            self.__balance = amount   # setter with validation
        else:
            print("Balance cannot be negative!")
```

Usage

```
acc = BankAccount("Riya", 1000)
print(acc.get_balance())      # 1000
acc.set_balance(1500)
print(acc.get_balance())      # 1500

# Trying direct access
print(acc.__balance)
# Error: AttributeError
```

4. Key Points

- ★ All attributes are private (__).
- ★ Data can be accessed or modified only through public methods.
- ★ Prevents accidental modification of data.
- ★ Provides maximum security, control and maintainability.

Hide Data
Protect Data
Control Data

Remember : Tightly Encapsulated Class = All data private (__)
Access only through methods (getters/setters).

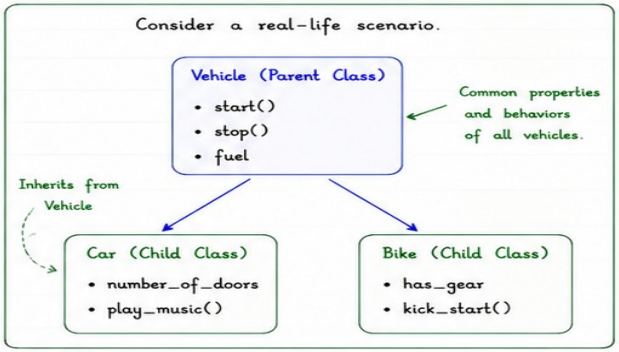
Inheritance in Python (OOP)

Date : 01/06/2026
Page No. : 1

1. What is Inheritance ?

- Inheritance is a mechanism in OOP that allows a new class (child class) to acquire the properties and behaviors (attributes and methods) of an existing class (parent class).
- The child class can also have its own attributes and methods in addition to those inherited from the parent class.
- It promotes code reusability, extensibility and maintainability.

2. Simple Example (Real Life)



Car and Bike inherit the features of Vehicle and can also have their own features.

3. Need of Inheritance

Code Reusability	We can reuse the existing code (methods and attributes) of a class in a new class.
Reduces Redundancy	Avoids rewriting the same code again and again.
Extensibility	We can add new features in the child class without modifying the parent class.
Easy Maintenance	Changes in the parent class are automatically available to all child classes.
Supports Real-Life Hierarchy	Helps to model real-world relationships (e.g., Is-A relationship).

Remember

- ★ Inheritance represents an "Is-A" relationship.
- ★ Child class is a specialized version of the parent class.

e.g.
Car IS-A Vehicle
Bike IS-A Vehicle

Inheritance in Python (OOP)

Date : 01/06/2026
Page No. : 2

1. Why is Duplication of Code Bad ?

When we write the same code (methods/attributes) again and again in multiple classes.

1. Hard to Maintain
If we need to change the code, we have to change it in all classes, which is time consuming and error-prone.

2. Increases Chances of Error
If we forget to update the code in any class, it may cause inconsistent behavior.

3. Wastes Code Space
Duplication increases the size of the code and makes it bulky.

4. Difficult to Extend
Adding new features or improvements becomes difficult when code is duplicated in many places.

Without Inheritance (Duplication)

Class Car

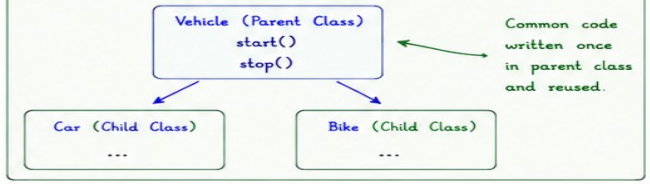
```
def start(self):
    print("Car started")
def stop(self):
    print("Car stopped")
    ...
```

Class Bike

```
def start(self):
    print("Bike started")
def stop(self):
    print("Bike stopped")
    ...
```

Same code (start and stop) is written again in both classes → Duplication!

With Inheritance (No Duplication)



2. Types of Relationship in OOP

There are 4 main types of relationships between classes.

1. Association (Has-A)	2. Aggregation (Has-A)	3. Composition (Has-A)	4. Inheritance (Is-A)
Student → Course	Department ◇→ Employee	House ◆→ Room	Parent Class → Child Class 1, Child Class 2
One class uses another class. They are independent but one is connected to another. Example: Student has a Course.	A special form of association. Whole and part are independent. If whole is destroyed, part can still exist. Example: Department has Employees.	A strong form of aggregation. Part cannot exist without the whole. If whole is destroyed, part is also destroyed. Example: House has Rooms.	One class is a specialized version of another class. Child (derived) class inherits properties and behaviors of parent (base) class. Example: Car is a Vehicle.

Remember

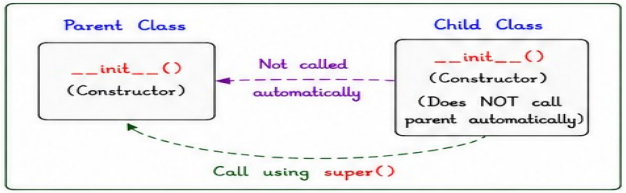
- ★ Duplicates make code harder to maintain, error-prone and bulky.
- ★ Inheritance helps in code reusability and better organization.

Inheritance in Python (OOP)

Date : 01/06/2026
Page No. : 3

1. Constructor Inheritance

- In Python, constructors are not inherited.
- When we create an object of the child class, only the child class constructor is called.
- If the parent class constructor needs to be used, we must call it explicitly using super().



Example

```
class Parent:
    def __init__(self, name):
        self.name = name                # Parent constructor
        print("Parent constructor called")

class Child(Parent):
    def __init__(self, age):            # Child constructor
        # super().__init__()  # <-- needed to call parent constructor
        self.age = age
        print("Child constructor called")
```

Without super()

```
obj = Child(20)
print(obj.age)
print(obj.name)   # AttributeError
```

Output:

```
Child constructor called
20
Traceback (most recent call last):
  ...
AttributeError: 'Child' object has no attribute 'name'
```

```
class Child(Parent):
    def __init__(self, name, age):     # Calling parent constructor
        super().__init__(name)
        self.age = age
        print("Child constructor called")
```

With super()

```
obj = Child("Riya", 20)
print(obj.name)
print(obj.age)
```

Output:

```
Parent constructor called
Child constructor called
Riya
20
```

Use super().__init__(...) to reuse and initialize attributes of the parent class.

2. AttributeError – The Dangerous Case

- If we forget to call the parent constructor (using super()), the attributes defined in the parent class are not created.
- When we try to access those attributes, Python raises AttributeError.
- This is a common and dangerous mistake in inheritance.

Dangerous Case
Parent attributes are missing because parent constructor was not called.
Accessing them → AttributeError

```
class Parent:
    def __init__(self):
        self.value = 100
        print("Parent __init__ called")

class Child(Parent):
    def __init__(self):
        # super().__init__() # Forgot!
        self.data = 50
        print("Child __init__ called")

obj = Child()
print(obj.data)      # Works
print(obj.value)     # AttributeError!
```

Output:

```
Child __init__ called
50
Traceback (most recent call last):
  ...
AttributeError: 'Child' object has no attribute 'value'
```

Always call super() in child constructor when parent has important initialization.

Remember

- Constructors are not inherited automatically.
- Forgetting super().__init__() can lead to AttributeError at runtime.

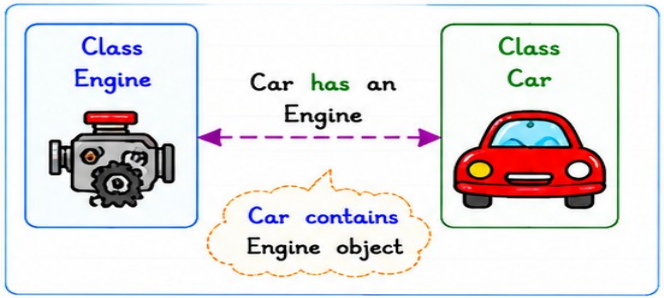
Has-A Relationship in Python (OOP)

Date : 01/06/2026
Page No. : 4

1. What is Has-A Relationship ?

- Has-A is a type of association.
- One class has an object of another class as its attribute/part.
- It represents "owns-a" or "contains-a" relationship.
- Used for code reusability and better modularity.

2. Simple Diagram (Has-A)



3. Types of Has-A Relationship

There are mainly 3 types of Has-A relationship.

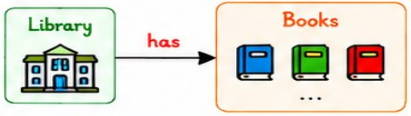
1 Simple Has-A

- A class has one object of another class.
- Most common case.


```
class Car:
  def __init__(self):
      self.engine = Engine()
```

2 Multiple Has-A

- A class has more than one object of another class.
- Used for collection of objects.




```
class Library:
  def __init__(self):
      self.books = []   # list of Book objects
```

3 Nested Has-A

- A class has an object of another class, which itself has other objects.
- Used in complex real-life cases.


```
class Employee:
  def __init__(self):
      self.address = Address()
```

4. Real-Life Examples

- A Car has an Engine.
- A Student has an Address.
- A Department has Employees.

- Car → has Engine
- Student → has Address
- Department → has Employees

Remember

Has-A shows relationship of ownership or containment, not inheritance.

Composition and Aggregation in Has-A Relationship

Date : 01/06/2026
Page No. : 5

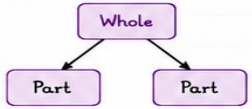
1. Introduction

- Has-A relationship can be of two special types: Composition and Aggregation.
- Both show "part-of" relationship, but they differ in ownership and lifetime of objects.
- They are stronger forms of association.

2. Visual Overview

Composition

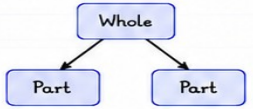
Strong ownership
(Part cannot exist without Whole)



If Whole is destroyed, Parts are also destroyed.

Aggregation

Weak ownership
(Part can exist without Whole)



If Whole is destroyed, Parts can still exist.

3. Key Difference (At a Glance)

Feature	Composition	Aggregation
Ownership	Strong ownership (Whole owns Parts)	Weak ownership (Whole uses Parts)
Lifetime	Parts die with the Whole	Parts can live independently
Relationship Strength	Stronger form of Has-A	Weaker form of Has-A
Dependency	Highly dependent	Less dependent
Example	House and Rooms	Person and Car

4. Real-Life Analogy

Composition Analogy

- A House is made up of Rooms.
- Rooms cannot exist without the House.
- If the House is demolished, Rooms also cease to exist.

House ◆——→ Room

Aggregation Analogy

- A Person has a Car.
- Car can exist even if the Person does not own it.
- If the Person stops using or sells the Car, Car still exists.

Person ◇——→ Car

Remember

★ Composition = Strong Has-A (Part cannot live without Whole)
★ Aggregation = Weak Has-A (Part can live without Whole)

e.g.

Composition vs Aggregation Programs in Python (Has-A Relationship)

Date : 01/06/2026
Page No. : 6

★ Below are simple Python programs to understand Composition and Aggregation.

1 Composition (Strong Has-A)

- House is made up of Rooms.
- If House is destroyed, Rooms also get destroyed.

```
class Room:
    def __init__(self, name):
        self.name = name

    def show(self):
        print(f"Room: {self.name}")

class House:
    def __init__(self):
        # Composition: House owns Room objects
        self.rooms = [Room("Bedroom"), Room("Kitchen"), Room("Hall")]

    def show_rooms(self):
        print("House has the following rooms:")
        for room in self.rooms:
            room.show()

    def __del__(self):
        # When House is destroyed, Rooms are also destroyed
        print("House is destroyed. Rooms are also destroyed.")
```

Output:

```
House has the following rooms:
Room: Bedroom
Room: Kitchen
Room: Hall

House is destroyed. Rooms are also destroyed.
```

Rooms cease to exist!

2 Aggregation (Weak Has-A)

- Person has a Car.
- Car can exist even if Person is gone.

```
class Car:
    def __init__(self, brand):
        self.namd = brand

    def show(self):
        print(f"Car: {self.brand}")

class Person:
    def __init__(self, name, car):
        # Aggregation: Person uses Car object
        self.name = name
        self.car = car

    def show(self):
        print(f"Person: {self.name}")
        self.car.show()

# Create Car object separately
c = Car("Honda")

# Create Person and give reference of Car
p = Person("Riya", c)

p.show()

# Even after deleting Person, Car still exists
del p
print("Car still exists after Person is deleted.")
c.show()
```

Output:

```
Person: Riya
Car: Honda
Car still exists after Person is deleted.
Car: Honda
```

Car still exists!

- Composition = Strong ownership (Whole owns Parts).
- Aggregation = Weak ownership (Whole uses Parts).
- In Composition, parts cannot live without the whole.
- In Aggregation, parts can live independently.

- ✔ Composition → "Can't live without" (Strong Has-A)
- ✔ Aggregation → "Can live without" (Weak Has-A)

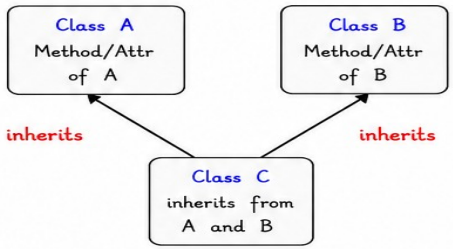
Multiple Inheritance and MRO in Python (OOP)

Date : 01/06/2026
Page No. : 7

1. What is Multiple Inheritance ?

- Multiple Inheritance is a feature in which a class can inherit from more than one parent class.
- It allows a child class to inherit attributes and methods from multiple base classes.
- It helps in code reusability and combining functionalities from different classes.

2. Simple Example



3. Syntax

```
class Parent1:
    pass
class Parent2:
    pass
class Child(Parent1, Parent2):
    pass
```

- Child class inherits features from both Parent1 and Parent2.
- The order of parents is important and is used in Method Resolution Order (MRO).
- Python supports any number of parent classes.

4. Why Multiple Inheritance ?

Code Reusability	Combining Features	Real-life Modeling	Flexibility
Reuse methods and attributes from multiple classes.	Combine functionalities from different classes in a single class.	Helps in modeling real-world entities that have multiple roles.	Provides more flexible and scalable designs.

5. Method Resolution Order (MRO) – Introduction

- When a child class inherits from multiple parent classes and two or more parents have the same method or attribute name, Python gets confused about which one to use.
- MRO (Method Resolution Order) is the order in which Python looks for a method or attribute.
- Python follows C3 Linearization algorithm to determine a consistent order.

Key Point
MRO ensures that Python always follows a specific and predictable order.

Remember

- Multiple Inheritance = One child class, multiple parent classes.
- MRO decides the order in which methods are searched.
- Use multiple inheritance carefully to keep code clean and understandable.

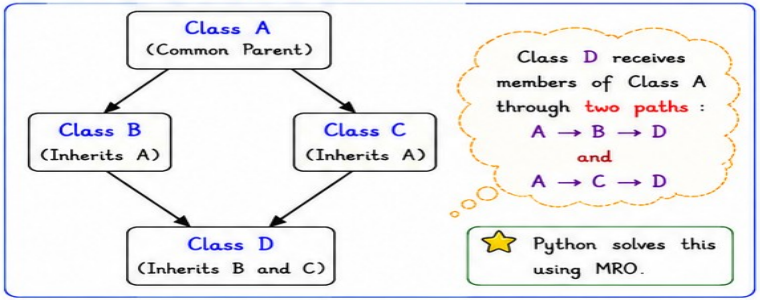
Multiple Inheritance – Diamond Problem and MRO in Python

Date : 01/06/2026
Page No. : 7

1. The Diamond Problem

- Occurs when a class inherits from two classes that have a common parent.
- The child class gets the same method from the common parent class through two different paths.
- Which one should Python use?
- This ambiguity is called the Diamond Problem.

2. Diamond Problem – Example



3. How Python Solves It – MRO

- Python uses MRO (Method Resolution Order) to decide the order in which base classes are searched.
- It follows C3 Linearization Algorithm.
- It ensures a consistent and predictable order so that each class is searched only once.

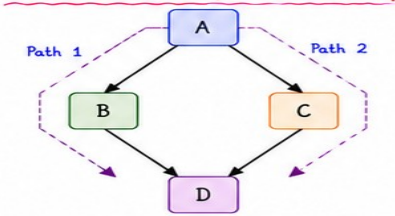
4. Example with Code

```
class A:
    def show(self):
        print("A")
class B(A):
    def show(self):
        print("B")
class C(A):
    def show(self):
        print("C")
class D(B, C):      # D inherits from B and C
    pass
```

```
d = D()
print(d.show())
print(D.mro())

# Output
B
[__main__.D,
 __main__.B,
 __main__.C,
 __main__.A,
 object]
```

5. MRO Order for the Example Above



MRO of D

D → B → C → A → object

- Python first looks in D.
- Then in B.
- Then in C.
- Then in A.
- Finally in object.

6. Key Points

- MRO avoids ambiguity in multiple inheritance.
- It ensures each class is visited once.
- Python follows C3 Linearization.
- Use super() to write cooperative multiple inheritance code.

Remember

- Diamond Problem occurs due to multiple paths to the same parent class.
- MRO (Method Resolution Order) defines the order in which classes are searched.
- Python uses C3 Linearization Algorithm to calculate MRO.

Method Signature and Introduction to Polymorphism in Python

Date : 01/06/2026
Page No. : 8

1. Method Signature

- The method signature is the combination of method name and the number, type and order of parameters it accepts.
- It does not include the return type.
- In Python, two methods have the same signature if they have the same name and same parameters (same number, type and order).

2. Examples of Method Signature

Method Definition	Signature
`def add(self, a, b):` `  pass`	`add(self, a, b)` `(3 parameters)`
`def display(self):` `  pass`	`display(self)` `(1 parameter)`

`def update(self, name, age=18):`
`  pass`
	`update(self, name, age)`
	`(3 parameters)`
	`# default value does not change the signature`

`def calc(self, x, y, z=0, *args):`
`  pass`
	`calc(self, x, y, z, *args)`
	`(variable length)`
	`# *args is treated as parameter`

3. Importance of Method Signature

- Helps the interpreter to identify the correct method to call.
- Essential for Method Overriding in Inheritance.
- Forms the base for achieving Polymorphism.

4. Introduction to Polymorphism

- Polymorphism means "many forms".
- It allows objects of different classes to be treated as objects of a common base class.
- The same method name can behave differently for different objects.
- Achieved in Python through:
  1. Method Overriding (Runtime Polymorphism)
  2. Method Overloading (Not directly supported, achieved using default args / *args / **kwargs)

5. Polymorphism Example (Method Overriding)

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

class Cat(Animal):
    def speak(self):
        print("Cat meows")
```

The same method call `obj_speak()` behaves differently based on the actual object.

Polymorphic Behavior

```
def call_speak(obj):
    obj.speak()

a = Animal()
d = Dog()
c = Cat()

call_speak(a)    # Animal speaks
call_speak(d)    # Dog barks
call_speak(c)    # Cat meows
```

6. Key Points

- Method signature = Method name + Parameters (number, type, order)
- Polymorphism allows flexible and extensible code.
- Promotes code reusability and easier maintenance.
- In Python, polymorphism is mainly achieved using method overriding.

Remember

- Method Signature helps in identifying methods.
- Polymorphism provides flexibility by the "one interface, many implementations" concept.
- Always override methods with the same signature in inheritance.

Method Overloading in Python

Date : 01/06/2026
Page No. : 10

1. What is Method Overloading ?

- Method Overloading means having multiple methods with the same name but different parameters (number, type or order).
- It allows a class to provide different implementations of the same method name.
- It improves code readability and flexibility.

2. Why Method Overloading is Useful ?

	Same operation can be performed in different ways.
	Increases code flexibility and reusability.
</>	Provides a clean and user-friendly interface.

3. Method Overloading vs Method Overriding

Feature	Overloading	Overriding
Definition	Same method name, different parameters	Same method name, same parameters
Where it occurs	In the same class	In parent and child class
Purpose	To perform similar operation in different ways	To provide new implementation
Binding	Compile-time (Static Binding)	Run-time (Dynamic Binding)
Supported in Python	Yes (using default arguments / *args / **kwargs)	Yes

4. How Python Achieves Method Overloading ?

- Python does not support traditional method overloading like Java or C++.
- But we can achieve method overloading using:
  1. Default Arguments
  2. Variable Length Arguments (*args)
  3. Keyword Variable Length Arguments (**kwargs)
- These allow a single method to accept different number or types of arguments.

5. Example – Method Overloading in Python

```
class Math:
    def add(self, a, b=None, c=None):
        if b is not None and c is not None:
            return a + b + c
        elif b is not None:
            return a + b
        else:
            return a

obj = Math()
print("add(5) =", obj.add(5))
print("add(5, 10) =", obj.add(5, 10))
print("add(5, 10, 15) =", obj.add(5, 10, 15))
```

Output:
```
add(5) = 5
add(5, 10) = 15
add(5, 10, 15) = 30
```

The same method name add() works with different number of arguments.

6. Example Using *args

```
class Math:
    def add(self, *args):
        result = 0
        for num in args:
            result += num
        return result

obj = Math()
print("add(5) =", obj.add(5))
print("add(5, 10) =", obj.add(5, 10))
print("add(5, 10, 15, 20) =",
    obj.add(5, 10, 15, 20))
```

Output:
```
add(5) = 5
add(5, 10) = 15
add(5, 10, 15, 20) = 50
```

*args allows the method to accept any number of positional arguments.

7. Example Using **kwargs

```
class Student:
    def info(self, **kwargs):
        for key, value in kwargs.items():
            print(f"{key} : {value}")

s = Student()
s.info(name="Riya", age=21, course="Python")
```

Output:
```
name : Riya
age : 21
course : Python
```

**kwargs allows the method to accept any number of keyword arguments.

8. Key Points

- Method Overloading = Same method name, different parameters.
- Python achieves overloading using default args, *args, **kwargs.
- It improves flexibility, usability and code cleanliness.
- Overloading is different from Overriding.

Remember

- Overloading provides multiple ways to call the same method.
- Python uses default arguments, *args and **kwargs to achieve overloading.
- Use method overloading to make your code more flexible and user-friendly.

Method Overriding in Python

Date : 01/06/2026
Page No. : 11

1. What is Method Overriding ?

- Method Overriding is a feature in OOPs where a child class provides a specific implementation of a method that is already defined in its parent class.
- The method in the child class has the same name, same parameters (method signature) and same return type as in the parent class.
- The child class method overrides (replaces) the parent class method.

2. Why Method Overriding is Used ?

Runtime Polymorphism	Specific Behavior	Code Reusability	Extensibility
			
It enables runtime polymorphism (Method Overriding is a key part of it).	The child class can provide its own specific implementation.	Promotes code reusability in inheritance.	Makes code easier to extend and maintain.

3. How It Works ?

- When the method is called using the child class object, Python looks for that method in the child class first.
- If found, it executes the child class method.
- If not found, it looks in the parent class.
- This is decided at runtime (hence, Runtime Polymorphism).

4. Example – Method Overriding in Python

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    # Overriding speak() method of parent class
    def speak(self):
        print("Dog barks")

a = Animal()
d = Dog()
a.speak()      # Calls Animal's speak()
d.speak()      # Calls Dog's speak()
```

Output:
Animal makes a sound
Dog barks

★ Dog class provides its own version of speak() method which overrides the speak() method of Animal class.

5. Method Overriding with Inheritance

```
class Animal
(Parent Class)
def speak(self):
    print("Animal
    makes a sound")
```

Inherits

```
class Dog(Animal)
    (Child Class)
def speak(self):
    print("Dog barks")
```

★ Dog inherits the speak() method from Animal. But it provides its own implementation.

6. Accessing Parent Class Method

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):
        print("Dog barks")
        # Calling parent class method
        super().speak()   # or: Animal.speak(self)

d = Dog()
d.speak()
```

Output:
Dog barks
Animal makes a sound

★ Using super() we can call the method of parent class inside child class method.

7. Key Points

1	2	3	4	5
Method overriding occurs in inheritance.	Method name and parameters must be the same.	Return type should be the same or compatible.	It is achieved at runtime (Runtime Polymorphism).	It helps in providing specific behavior in child class.

★ Overriding = Same method signature in child class with different implementation.
★ Overloading = Same method name in same class with different parameters.
★ Overriding supports Polymorphism (One interface, multiple implementations).

Operator Overloading in Python

Date : 01/06/2026
Page No. : 12

1. What is Operator Overloading ?

- Operator Overloading is a feature in Python that allows us to define the meaning of operators (+, -, *, ==, etc.) for objects of a user-defined class.
- It works by defining special (dunder) methods in the class.
- It makes code more readable and intuitive.
- It does not change the operator itself, only its behavior for user-defined objects.

2. Built-in Operators That Can Be Overloaded

Category	Operators	Special Method
Arithmetic	+ - * / // % **	__add__, __sub__, __mul__, __truediv__, __floordiv__, __mod__, __pow__
Comparison	== != < <= > >=	__eq__, __ne__, __lt__, __le__, __gt__, __ge__
Assignment	= += -= *= /= //= %= **=	__assign__, __iadd__, __isub__, __imul__, __itruediv__, etc.
Bitwise	& | ^ ~ << >>	__and__, __or__, __xor__, __invert__, __lshift__, __rshift__

3. How It Works ?

- When an operator is used with objects, Python internally calls the corresponding special method.
- If that method is defined in the class, it is executed.
- Otherwise, Python raises an error.

4. Example – Overloading + Operator

```
class Complex:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    # overloading + operator
    def __add__(self, other):
        real = self.real + other.real
        imag = self.imag + other.imag
        return Complex(real, imag)
    def __str__(self):
        return f"{self.real} + {self.imag}i"

c1 = Complex(2, 3)
c2 = Complex(4, 5)
c3 = c1 + c2      # Calls __add__() method
print(c3)
```

Output:
6 + 8i

Here, + operator is overloaded using __add__() method.

5. Common Arithmetic Operator Overloading

Operator	Meaning	Special Method
+	Addition	__add__(self, other)
-	Subtraction	__sub__(self, other)
*	Multiplication	__mul__(self, other)
/	True Division	__truediv__(self, other)
//	Floor Division	__floordiv__(self, other)
%	Modulus	__mod__(self, other)
**	Power	__pow__(self, other)
-x	Unary Minus	__neg__(self)

6. Example – Overloading == Operator

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

v1 = Vector(3, 4)
v2 = Vector(3, 4)
v3 = Vector(1, 2)

print(v1 == v2)    # Calls __eq__() -> True
print(v1 == v3)    # Calls __eq__() -> False
```

Output:
True
False

The == operator is overloaded using __eq__() method.

7. Key Points

- Operator overloading makes code more natural and expressive.
- It is achieved by defining special (dunder) methods.
- It improves readability and works like built-in types.
- It does not change the operator, only its behavior.
- Helps in creating class objects that behave like native Python objects.

Remember

- Operator Overloading = Defining custom behavior of operators for user-defined classes.
- Implemented using special methods (dunder methods).
- Makes our classes act like built-in types.

Whenever an operation on two objects should produce another object of the same class, return a new object:

Instance Methods in Python

Date : 01/06/2026
Page No. : 13

1. What is an Instance Method ?

- An **instance method** is a function defined inside a class that operates on the **instance (object)** of the class.
- It must have at least one parameter: **self**, which refers to the current instance.
- Instance methods can access and modify **instance variables**.
- They are called on objects, not on the class.

2. Syntax of Instance Method

```
class ClassName:
    def method_name(self, arg1, arg2, ...):
        # method body
```

Explanation:

- **self** – refers to the current object.
- **method_name** – name of the method.
- **arg1, arg2** – other parameters (optional).
- The method works with the data of the object.

3. Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

s1 = Student("Riya", 20)
s1.display()   # Output: Name: Riya, Age: 20
```

Here, display() is an instance method. It is called on the object s1 and uses its data (name, age).

4. How It Works ?

- When you call **s1.display()**, Python automatically passes **s1** as the first argument (**self**).
- Inside the method, **self** refers to **s1**.
- The method can access all instance variables using **self**.
- Each object has its own copy of instance variables, but they share the **same method definition**.

5. Accessing Instance Variables

- Inside an instance method, use **self.variable_name** to access instance variables.
- You can also modify them.

6. Modifying Instance Variables

```
class Counter:
    def __init__(self):
        self.count = 0

    def increment(self):
        self.count += 1   # Modifying instance variable
        print("Count:", self.count)

c = Counter()
c.increment()   # Output: Count: 1
c.increment()   # Output: Count: 2
```

Each object keeps its own value of count.

7. Multiple Objects – Separate Data

```
s1 = Student("Riya", 20)
s2 = Student("Aman", 22)

s1.display()   # Name: Riya, Age: 20
s2.display()   # Name: Aman, Age: 22
```

Both objects use the same method display(), but operate on their own data.

8. Key Points

- Instance methods work with objects (instances).
- **self** refers to the current object.
- They can access and modify **instance variables**.
- They are called using **object_name.method_name()**.
- Each object has its own data, but shares the same methods.

Remember

- Instance Method = Works on instance (object) and uses instance variables.
- Always include **self** as the first parameter.
- Call instance methods on **objects**, not on the **class**.

Class Methods in Python

Date : 01/06/2026
Page No. : 14

1. What is a Class Method ?

- A class method is a method that is bound to the class and not to any instance.
- It takes cls as the first parameter, which refers to the class itself.
- It can access and modify class variables.
- It is defined using the @classmethod decorator.
- It is called on the class, not on an object (though it can be called on an object too).

2. Syntax of Class Method

```
class ClassName:
    class_variable = value

    @classmethod
    def method_name(cls, arg1, arg2, ...):
        # method body
```

Explanation:

- @classmethod – declares the method as a class method.
- cls – refers to the class.
- The method works with the class and its data.
- It can access/modify class variables, not instance variables directly.

3. Example

```
class Student:
    school = "ABC School"   # class variable

    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def change_school(cls, new_school):
        cls.school = new_school

s1 = Student("Riya", 20)
Student.change_school("XYZ School")
print(Student.school)   # Output: XYZ School
```

change_school() is a class method. It changes the class variable school for all students.

4. How It Works ?

- When you call Student.change_school("XYZ School"), Python passes the class Student as the first argument (cls).
- Inside the method, cls refers to Student.
- cls.school accesses the class variable.
- When cls.school is changed, it affects all objects (current and future) of the class.

5. Accessing / Calling Class Method

- Preferred way: Call using the class name.
  `ClassName.class_method(...)`
- You can also call it using an object (not recommended, but allowed).
  `object_name.class_method(...)`

6. Class Method vs Instance Method

Class Method	Instance Method
Works with the class.	Works with the instance (object).
First parameter is cls.	First parameter is self.
Can access/modify class variables.	Can access/modify instance variables.
Called on the class (ClassName.method()).	Called on the object (object.method()).
Defined with @classmethod.	No decorator.
Cannot access instance variables directly.	Can access instance variables directly.

7. Practical Example

```
class Counter:
    count = 0     # class variable

    def __init__(self):
        Counter.count += 1

    @classmethod
    def get_count(cls):
        return cls.count
```

```
c1 = Counter()
c2 = Counter()
c3 = Counter()

print(Counter.get_count())
# Output: 3

print(c1.get_count())
# Output: 3
```

get_count() is a class method that returns the total number of objects created.

8. Key Points

- Class methods are bound to the class, not to any instance.
- Always use cls as the first parameter.
- Defined using the @classmethod decorator.
- Can access and modify class variables.
- Called using the class name (recommended).
- Useful for factory methods, utility functions, and operations related to the class as a whole.

Remember

- Use class methods when the method is related to the class itself, not to any particular object.
- They help in modifying class-level data and creating alternative constructors.
- Class methods improve code organization and promote better design.

Static Methods in Python

Date : 01/06/2026
Page No. : 15

1. What is a Static Method ?

- A static method is a method that belongs to the class but does not require any instance (object) or class (cls) to access it.
- It does not take self or cls as the first parameter.
- It is defined using the @staticmethod decorator.
- It cannot access or modify instance or class variables directly.
- It behaves like a regular function, but is placed inside the class for logical grouping.

2. Syntax of Static Method

```
class ClassName:
    @staticmethod
    def method_name(arg1, arg2, ...):
        # method body
```

Explanation:

- @staticmethod – declares the method as a static method.
- No self or cls parameter.
- method_name – name of the static method.
- arg1, arg2 – optional parameters.
- It is called on the class, not on an object.

3. Example

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def is_even(n):
        return n % 2 == 0
```

add() and is_even() are static methods inside the MathUtils class.

4. How It Works ?

- When you call MathUtils.add(10, 20), Python directly calls the method. No object is created.
- No self or cls is passed automatically.
- The method works only with the data passed as arguments.
- It cannot access instance variables or class variables directly.

5. Calling Static Method

- Preferred way: Call using the class name.

```
ClassName.static_method(...)
```

- You can also call it using an object (not recommended, but allowed).

```
object_name.static_method(...)
```

6. Static Method vs Instance Method vs Class Method

	Static Method	Instance Method	Class Method
Belongs to	Class	Object (instance)	Class
First parameter	None	self	cls
Decorator	@staticmethod	None	@classmethod
Access instance variables	No	Yes	No (directly)
Access class variables	No (directly)	Yes (via class name)	Yes
Called on	Class (preferred)	Object	Class (preferred)
Use case	Utility / Helper functions	Work with object's data	Work with class-level data

7. Practical Example

```
class Student:
    school = "ABC School"    # class variable

    def __init__(self, name):
        self.name = name     # instance variable

    @staticmethod
    def is_valid_age(age):
        return 5 <= age <= 18
```

```
print(Student.is_valid_age(15))    # True
print(Student.is_valid_age(25))    # False

s1 = Student("Riya")
print(Student.is_valid_age(10))    # True
print(s1.is_valid_age(10))         # True
```

is_valid_age() works the same on class or object because it does not depend on any instance or class data.

8. Key Points

- Static methods do not take self or cls.
- Defined using @staticmethod decorator.
- Called using the class name (preferred).
- Cannot access instance variables or class variables directly.
- Useful for utility functions that are logically related to the class.

Remember

- Use static methods for general utility functions.
- They are independent of both class state and instance state.
- They help in organizing code and improving readability.

Instance, Class & Static Methods in Python

Key Points

- **Instance Method** → Works with object (self).
- **Class Method** → Works with class (cls).
- **Static Method** → Works neither with object nor class.

Syntax

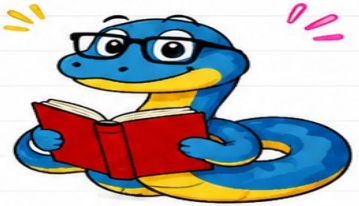
```
Instance Method → def func(self):
Class Method    → @classmethod
                  def func(cls):
Static Method   → @staticmethod
                  def func():
```

Simplified Summary Table

Method Type	Access Instance Variable	Access Class Variable
Instance Method (self)	✓ self.var	✓ ClassName.var or self.__class__.var
Class Method (cls)	✗ Directly not possible	✓ cls.var
Static Method	✗ Directly not possible	✓ ClassName.var

Remember:

- self → Object
- cls → Class
- Static → Neither self nor cls



Use self for object, cls for class and Static for utility function.